

STEP 3

Feature engineering and preprocessing pipeline

Turn raw columns into model-ready numbers, inside one object that is saved with the model.

Engineered features

Feature	Formula	Why
Month	month of Date	Seasonality
Day_Of_Week	0 = Monday ... 6 = Sunday	Weekly demand pattern
Is_Weekend	Day_Of_Week $\geq$ 5	Weekend effect
Order_To_Demand_Ratio	Purchase_Quantity / Daily_Demand	Over-ordering is the main driver of waste (see Figure 2, right)

The pipeline

Everything is chained in one scikit-learn Pipeline:

```
Pipeline([
    ("features", FunctionTransformer(add_features)),          # Month, Day_Of_Week, ratio ...
    ("preprocess", ColumnTransformer([
        ("num", SimpleImputer(median) -> StandardScaler,      numeric columns),
        ("cat", SimpleImputer(most_frequent)
            -> OneHotEncoder(handle_unknown="ignore"),          categorical columns),
    ])),
    ("model", <classifier>),
])
```

How missing values and outliers are handled

- **Missing numbers** (Shelf_Life_Days, Storage_Temperature, Daily_Demand) are filled with the training median. Missing Weather is filled with the most frequent value.
- **Unknown categories** at prediction time are ignored by the one-hot encoder instead of crashing (the API already rejects them, so this is a second safety net).
- **Outliers** in Purchase_Quantity are kept. They are real bulk orders, and tree models are robust to them.
- **Class imbalance** is handled with `class_weight="balanced"` so the rarer High class is not ignored.

Why it matters

Because preprocessing is part of the saved model, the API only needs raw fields. There is no second copy of the feature code that could get out of sync with training. This mismatch, called **training-serving skew**, is one of the most common production ML bugs.